SERVER NETWORK LATENCY ANALYSIS USING DIJKSTRA'S ALGORITHM IN C++

1. TITLE

Server Network Latency Analysis Using Dijkstra's Algorithm in C++

2. PROBLEM STATEMENT

In computer networks, multiple servers are connected through network links, and each link has a specific latency. Latency represents the time taken for data to travel between connected servers. Finding the minimum latency between servers is important for efficient network communication and routing.

The aim of this project is to develop a C++ program that represents servers as vertices and network links as weighted edges in a graph. The weight of each edge represents the latency between two servers. The program applies Dijkstra's Algorithm to calculate the minimum latency from a selected source server to all reachable servers, ranks servers in ascending order of minimum latency, and identifies servers that cannot be reached from the source server.

3. OBJECTIVES

1. To represent a computer network using a weighted graph.
2. To represent servers as vertices and network links as weighted edges.
3. To implement Dijkstra's Algorithm using C++.
4. To calculate the minimum latency from a selected source server to all reachable servers.
5. To identify unreachable servers in the network.
6. To rank servers according to their minimum latency in ascending order.
7. To understand graph data structures, priority queues, and shortest-path algorithms.
8. To demonstrate how shortest-path algorithms can support efficient network routing.

4. ALGORITHM

1. Start the program.
2. Input the number of servers and the number of network links.
3. Create a weighted graph using an adjacency list. Represent each server as a vertex and each network link as an edge with its latency.
4. Input the source server from which minimum latency must be calculated.
5. Initialize the source server's distance to 0 and all other distances to infinity.
6. Insert the source server into a minimum priority queue.
7. Remove the server with the smallest recorded distance from the priority queue.
8. Examine all neighboring servers connected to the selected server.
9. Calculate the possible new latency to each neighbor by adding the current distance and the link latency.

Scanned with OKEN Scanner

10. If the new latency is smaller than the recorded distance, update the distance and insert the updated value into the priority queue.
11. Repeat the processing steps until the priority queue becomes empty.
12. Collect reachable servers and sort them in ascending order of minimum latency.
13. Display the ranked servers with their minimum latencies and list unreachable servers separately.
14. Stop the program.

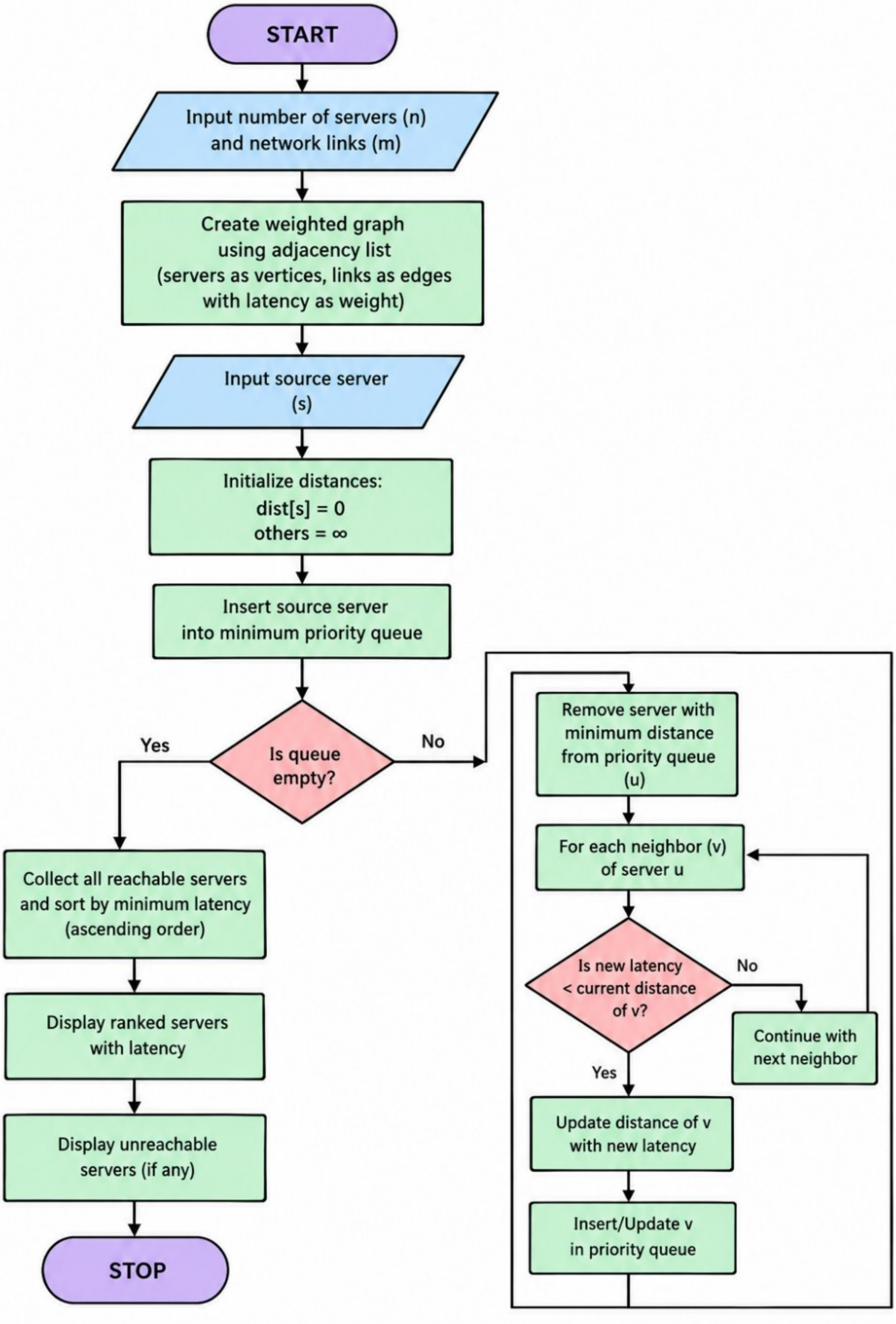
Note: Dijkstra's Algorithm requires non-negative edge weights.

Scanned with OKEN Scanner

5. FLOWCHART

Flowchart for Server Network Latency Analysis using Dijkstra’s Algorithm

Scanned with OKEN Scanner



Scanned with OKEN Scanner

6. CONCLUSION

The Server Network Latency Analysis project demonstrates the implementation of Dijkstra's Algorithm using C++. Servers are represented as vertices and network links as weighted edges, where each weight represents latency. The algorithm calculates the minimum latency from a selected source server to all reachable servers, ranks them in ascending order of latency, and identifies unreachable servers. This project builds practical understanding of graph representation, priority queues, and shortest-path algorithms, and shows how these techniques can support network routing analysis and efficient communication.

Scanned with OKEN Scanner